PRAKTIKUM 4

Instalasi dan Menjalankan Apache Spark Standalone di Windows

1. Tujuan Praktikum

Setelah praktikum ini mahasiswa dapat:

- Menjalankan Apache Spark menggunakan Docker di Windows.
- Mengenal Spark Master, Spark Worker, Driver, Executor, dan Task.
- Membuat Spark cluster sederhana dengan 1 Master dan 2 Worker.
- Membuka Spark Master UI.
- Menjalankan spark-shell.
- Menjalankan aplikasi Spark sederhana.
- Memahami hubungan Spark dengan HDFS secara konseptual.

2. Konsep yang Dipakai

```
HDFS → menyimpan data

Spark → memproses data

Master → mengatur resource Spark Standalone

Worker → menyediakan resource untuk Spark

Driver → mengendalikan Spark Application

Executor → menjalankan task

Task → unit pekerjaan
```

Catatan: Pada praktikum ini kita menggunakan Spark Standalone. Cluster Manager yang digunakan adalah Spark Master, bukan YARN.

3. Aplikasi yang Harus Tersedia

Aplikasi	Fungsi	Status
Windows 10/11	Sistem operasi	Wajib
Docker Desktop	Menjalankan container	Wajib
Command Prompt (CMD)	Menjalankan perintah Docker	Wajib
Chrome/Edge	Membuka Spark UI	Wajib
VS Code	Membuat/mengedit file YAML	Opsional

Catatan: Spark tidak perlu di-install langsung ke Windows. Spark dijalankan di dalam Docker container.

4. Struktur yang Akan Dibuat

```
C:\bigdata\
└── spark-praktikum\
    └── docker-compose.spark.yml
```

Setelah container dijalankan:

```
Docker
├── spark-master
├── spark-worker-1
└── spark-worker-2
```

5. Langkah 1: Pastikan Docker Desktop Berjalan

1. Buka Docker Desktop.
2. Tunggu sampai Docker Engine berjalan.
3. Jangan lanjut jika Docker belum berjalan.

Verifikasi dari CMD:

```
docker --version

docker compose version
```

Catatan: Jika kedua perintah menampilkan versi, Docker siap digunakan.

6. Langkah 2 :Membuat Folder Praktikum

Buka Command Prompt (CMD)

```
cd C:\

mkdir bigdata

cd bigdata

mkdir spark-praktikum

cd spark-praktikum

dir
```

Catatan: Jika C:\bigdata sudah ada dari praktikum sebelumnya, langsung masuk ke folder tersebut.

7. Langkah 3 :Membuat File docker-compose.spark.yml

File ini memberi tahu Docker cara membuat Spark Master dan dua Spark Worker. Cara termudah adalah menggunakan Notepad.

```
notepad docker-compose.spark.yml
```

4. Jika Notepad bertanya apakah file baru akan dibuat, pilih Yes.
5. Salin seluruh isi berikut ke Notepad:

```
services:

  spark-master:

    image: apache/spark:3.5.6

    container_name: spark-master

    command: ["/opt/spark/bin/spark-class", "org.apache.spark.deploy.master.Master"]

    ports:

      - "7077:7077"

      - "8080:8080"



  spark-worker-1:

    image: apache/spark:3.5.6

    container_name: spark-worker-1

    depends_on:

      - spark-master

    command: ["/opt/spark/bin/spark-class", "org.apache.spark.deploy.worker.Worker",
"spark://spark-master:7077"]

    ports:

      - "8081:8081"



  spark-worker-2:

    image: apache/spark:3.5.6

    container_name: spark-worker-2

    depends_on:

      - spark-master

    command: ["/opt/spark/bin/spark-class", "org.apache.spark.deploy.worker.Worker",
"spark://spark-master:7077"]

    ports:

      - "8082:8081"
```

6. Di Notepad pilih File → Save.
7. Tutup Notepad.

Kembali ke CMD dan ketik:

```
dir
```

Harus terlihat file:

```
docker-compose.spark.yml
```

8. Langkah 4 :Memeriksa File Compose

```
type docker-compose.spark.yml
```

Catatan: Jika file tidak ditemukan, pastikan Anda berada di C:\bigdata\spark-praktikum dan nama file tepat.

9. Langkah 5 :Menjalankan Spark Cluster

```
docker compose -f docker-compose.spark.yml up -d
```

Catatan: Pertama kali dijalankan, Docker mungkin perlu mengunduh image Apache Spark.

10. Langkah 6 :Mengecek Container

```
docker ps
```

Harus terlihat:

```
spark-master

spark-worker-1

spark-worker-2
```

Ketiganya harus berstatus Up.

11. Jika Container Tidak Muncul

```
docker compose -f docker-compose.spark.yml ps

docker logs spark-master

docker logs spark-worker-1

docker logs spark-worker-2
```

12. Langkah 7 :Membuka Spark Master UI

Buka Chrome/Edge:

```
http://localhost:8080
```

Amati Master URL, Workers, CPU, dan Memory.

Catatan: Jika Master URL yang tampil berbeda, ikuti URL yang ditampilkan UI.

13. Langkah 8 :Membuka Spark Worker UI

```
http://localhost:8081

http://localhost:8082
```

Catatan: Port 8082 di Windows diarahkan ke port 8081 di container worker kedua.

14. Langkah 10 :Masuk ke Container Spark Master

```
docker exec -it spark-master bash
```

15. Langkah 11 :Mengecek Spark

```
/opt/spark/bin/spark-submit --version
```

16. Langkah 12 :Menjalankan Spark Shell

```
/opt/spark/bin/spark-shell --master spark://spark-master:7077
```

Catatan: Jika berhasil muncul prompt scala>. Spark-shell menggunakan Scala, tetapi praktikum ini hanya menggunakan kode sederhana.

17. Langkah 13 :Program Spark Pertama

```
val data = Seq(1, 2, 3, 4, 5)

val rdd = sc.parallelize(data)

rdd.collect()

Array(1, 2, 3, 4, 5)
```

18. Langkah 14 :Transformation Pertama

```
val hasil = rdd.map(x => x * 2)

hasil.collect()

Array(2, 4, 6, 8, 10)
```

map() adalah transformation. collect() adalah action atau perintah untuk eksekusi data.

19. Langkah 15 :Spark Application UI

```
http://localhost:4040
```

Amati Jobs, Stages, Executors, dan Environment.

Catatan: 4040 adalah UI aplikasi Spark yang sedang berjalan; 8080 adalah Spark Master UI.

20. Langkah 17 :Melihat Partition

```
rdd.getNumPartitions
```

```
val rdd4 = sc.parallelize(Seq(1,2,3,4,5,6,7,8), 4)

rdd4.getNumPartitions

rdd4.glom().collect()
```

Catatan: glom() membantu melihat isi masing-masing partition.

21. Langkah 18 :filter()

```
val genap = rdd4.filter(x => x % 2 == 0)

genap.collect()

2, 4, 6, 8
```

22. Langkah 19 :WordCount Sederhana

```
val teks = sc.parallelize(Seq("spark hadoop", "spark big data", "hadoop data"))

val words = teks.flatMap(line => line.split(" "))

val pairs = words.map(word => (word, 1))

val counts = pairs.reduceByKey((a,b) => a + b)

counts.collect()

teks → flatMap() → kata → map() → (kata,1) → reduceByKey() → (kata,jumlah)
```

23. Langkah 20 :Keluar dari Spark Shell

```
:quit

exit
```

24. Langkah 21 :Memastikan HDFS Aktif

```
docker ps
```

Pastikan container Hadoop dari praktikum sebelumnya aktif.

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum/hdfs/input
```

Catatan: Jika lokasi input berbeda pada cluster Anda, gunakan lokasi HDFS yang memang dibuat pada praktikum sebelumnya.

25. Langkah 22 :Membaca File HDFS dengan Spark

```
docker exec -it spark-master bash

/opt/spark/bin/spark-shell --master spark://spark-master:7077

val file = sc.textFile("hdfs://namenode:9000/praktikum/hdfs/input/input.txt")

file.count()
```

```
file.collect()
```

26. Langkah 23 :WordCount dari HDFS

```
val wordsHDFS = file.flatMap(line => line.split("\s+"))

val pairsHDFS = wordsHDFS.map(word => (word, 1))

val countsHDFS = pairsHDFS.reduceByKey((a,b) => a + b)

countsHDFS.collect()
```

27. Langkah 24 :Menghentikan Spark Cluster

```
:quit

exit

cd C:\bigdata\spark-praktikum

docker compose -f docker-compose.spark.yml down

docker ps
```

28. Spark Standalone vs Spark on YARN

Model	Cluster Manager	Alur
Spark Standalone	Spark Master	Driver → Master → Worker → Executor
Spark on YARN	YARN ResourceManager	Driver → ResourceManager → NodeManager/Container → Executor

Catatan: Praktikum ini sengaja menggunakan Spark Standalone agar mahasiswa memahami arsitektur Spark terlebih dahulu.

29. Tugas Praktikum

1. Buat dan jalankan Spark Standalone dengan 1 Master dan 2 Worker.
2. Screenshot Spark Master UI yang menunjukkan dua Worker.
3. Jalankan spark-shell.
4. Buat RDD menggunakan parallelize().
5. Tampilkan isi RDD menggunakan collect().
6. Tentukan jumlah partition.
7. Gunakan map() dan filter().
8. Buat WordCount dengan reduceByKey().
9. Baca input.txt dari HDFS menggunakan textFile().
10. Jalankan WordCount terhadap file HDFS.
11. Screenshot Spark Application UI.

30. Troubleshooting

Masalah	Solusi awal
docker compose tidak dikenali	Pastikan Docker Desktop aktif dan jalankan docker compose version.
docker-compose.spark.yml tidak ditemukan	Jalankan dir dan pastikan file berada di folder aktif.
spark-master tidak Up	Jalankan docker logs spark-master.
Worker tidak muncul	Periksa docker logs spark-worker-1 dan spark-worker-2.
Spark UI 8080 tidak bisa dibuka	Periksa docker ps dan port mapping.
Spark UI 4040 tidak muncul	Pastikan spark-shell masih berjalan.
HDFS file tidak ditemukan	Periksa lokasi HDFS dengan hdfs dfs -ls.
Spark tidak bisa membaca HDFS	Periksa hostname/port fs.defaultFS dan jaringan Docker.

S